File Assembly Step by Step — A Tutorial

This is the third, most detailed version of the guide on file assembly and collision resolution — written "for technical-college students": every term is introduced from scratch, every step of the algorithm is walked through with numbers, tables, and traces. The formal monograph is `AssemblyGuide.Academic.en.md`; the short engineering retelling is `AssemblyGuide.Engineer.en.md` (their section numbering matches each other; this tutorial uses its own, lesson-style numbering). General context — packet format, hashes, accumulation — is in `AlgorithmGuide.en.md`.

After reading this tutorial you should be able to explain in your own words: where sector versions come from, why assembly is verified by a single final hash, how "candidate #1" is chosen, what a choice point is, how exhaustive search (the "odometer"), heuristic rotation, and the volume subset search work, and why a forgery can never turn into an assembled file.

1. Vocabulary: the words we cannot do without

Term	What it actually is
Sector (volume)	One numbered piece of data, exactly 64 bytes. Numbers: `0 … T-1`. The first `N` numbers are data volumes (chunks of the file), the last `M` are ECC volumes (redundancy for recovering lost ones).
`N`	Number of data volumes. Not stored in the header; computed as `N = ceil(FileSize / 64)`, at least 1.
`M`	Number of ECC volumes (from the header). May be 0.
`T`	`T = N + M` — total number of volumes. Upper bound: 65,535.
Payload	The 64 bytes of a sector themselves: either a chunk of the file or ECC data.
Header	The 51-byte "cap" of a file: name, size, SHA-256 of the file, `M`.
`H3`	SHA-256 of the original file's contents (32 bytes, a header field). The sole judge of assembly.
`H5`	24-byte hash of the header; the seed from which sector hashes are derived.
Version	One concrete payload variant for one sector number.
Confirmation count	How many times this version arrived from the stream. Any version has ≥ 1.
Collision	More than one version for one number.
Tied head	The beginning of a version list: all leaders sharing the same maximal count.
Choice point	A sector whose head holds ≥ 2 versions. Its competing variants are what gets searched.
Selection (`selected`)	A set of "one payload per number, 0 through T−1". One candidate for a finished file.
Attempt	Checking one selection: direct assembly, on failure — RS recovery, then the final hash.
Reception slot	The accumulator of everything received about one file: header + version lists. The `ReceptionSlot` class.
Odometer	The exhaustive-search algorithm over combinations — like a trip meter: the rightmost digit spins fastest.
Rotation	The heuristic walk: all contested lists advance by one step simultaneously.
Exclusion mask	The set of volume numbers that a stage-3 attempt deliberately treats as erased.

2. What the assembler has on the table

The decoder has already scanned the stream and sorted what it received. Recall what a single packet — 75 bytes — looks like:

Field	Size, bytes	Offset	Meaning
Sector number	2	0	`0 … T−1`
Payload	64	2	a chunk of the file (data) or redundancy (ECC)
Sector hash (D3)	9	66	truncated SHA-256, 72 bits — a quick "was the packet corrupted" check

The header travels as its own packet: 51 bytes of fields + 24 bytes of `H5` — the same 75 bytes. Header fields:

Field	Size, bytes	Offset	Meaning
File name	14	0	fixed length, zero-padded
File size	3	14	little-endian; maximum 16,777,215 bytes (~16 MiB)
SHA-256 of the file (`H3`)	32	17	the authenticity judge
ECC volume count `M`	2	49	little-endian

In a textual stream each 75-byte packet is additionally Base64-encoded — exactly 100 characters.
So, at assembly time we have:

- the header → hence `N`, `M`, `T`, `FileSize`, and the reference hash `H3` are known;
- a pile of received sectors sorted by number. One number may hold several **different** payloads — these are versions, each with its own counter.

Everything that follows answers one question: which exact versions to pick so that the file assembles.

3. Why there is exactly one, final check

The assembly formula is dead simple:

```
buffer = payload[0] ‖ payload[1] ‖ … ‖ payload[N−1]      // N × 64 bytes
  result = buffer trimmed to FileSize bytes
  if SHA-256(result) == H3   →  the file is ready
  else                       →  this selection is wrong, try another
```

Three consequences worth learning like the multiplication table:

1. **The check does not depend on the reception process.** Neither counters, nor packet arrival order, nor heuristic decisions take part in the formula. There is one judge — SHA-256.
2. **The outcome is binary.** Either a file bit-for-bit equal to the original (proof: the hash matches), or an honest refusal (`null`). The state "almost assembled, ship it anyway" does not exist in principle.
3. **The final hash cannot be cheated.** That would require a SHA-256 preimage — practically infeasible today. The worst an attacker can do is force us to iterate over versions and waste time.

4. How versions accumulate: `AddSector` step by step

The heart of the accumulator is a dictionary "sector number → version list", and the list is **always** sorted by descending confirmation count (this is an invariant, covered by tests).

Receiving one sector copy (`AddSector(sectorNum, payload)`) works like this:

1. **Two entry checks.** Is the number within `0 … T-1`? Is the payload exactly 64 bytes? If not — the copy is silently discarded, `false` is returned.
2. **Search for an existing version.** The incoming payload is compared byte-by-byte with every version in the list.
3. **Found a match** → its counter gets `+1` (saturating: it never grows past `int.MaxValue`). The version then "bubbles up" — but only **strictly above** versions with a smaller counter. It is **never reordered past** versions with an equal counter: the order of equals is untouchable (this matters for rotation, section 11).
4. **No match** → a new version with counter 1 is appended to the end of the list. At the end — because every existing version already has a counter ≥ 1, so the sorting is not broken.

Example. Let sector #1 arrive in the order `P, P, Q, P, Q`. The list evolves like this:

Incoming copy	Action	List afterwards
`P`	no version matches → append, counter 1	`[P×1]`
`P`	matched `P` → counter 2	`[P×2]`
`Q`	matched nothing → new version at the end	`[P×2, Q×1]`
`P`	matched → counter 3	`[P×3, Q×1]`
`Q`	matched → counter 2; nothing to bubble past: `P×3` is ahead	`[P×3, Q×2]`

And here is bubbling in action. Suppose we had `[X×3, P×2]`:

Incoming copy	List afterwards	Why
`P`	`[X×3, P×3]`	counters became equal, but equals are not jumped over
`P`	`[P×4, X×3]`	`4 > 3` — the version bubbled into first place

Note: an ordinary repeat (the same payload again) is **not** a collision — it reinforces the truth. A collision is precisely *different* payloads under one number.

5. Where different versions of one number even come from

1. **A random collision of the 72-bit sector hash.** The probability is vanishingly small, but theoretically a "foreign" packet can pass the quick D3 check.
2. **Forgery.** `H5` is transmitted in the open, so anyone who sees the stream can generate "valid" sectors with arbitrary content. The entire version-and-search machinery is built for exactly this case.

Three layers work against forgery: counters (repeats strengthen the truth), the search over tied candidates, and the final hash (see section 3).

6. Candidate #1: the most confirmed versions

The first selection that assembly tries is built trivially: **the first element of every list** — i.e., the most confirmed versions. Uncontested sectors (a single version or a clear leader) create no choice at all: their contribution to every attempt is identical.

Example. `T = 4`, lists:

```
#0: [P0×5]          — one version, no choice
  #1: [P1×3, Q1×3]    — tie at the top: there is a choice
  #2: [P2×7, R2×2]    — clear leader: no choice
  #3: [P3×1, Q3×1]    — tie: there is a choice
```

Candidate #1: `[P0, P1, P2, P3]` — the first element everywhere.

7. The simplest case: no dispute

If no sector has a tied head of ≥ 2 versions, there is nothing to search: the selection is determined unambiguously. Assembly makes exactly one attempt (direct assembly, on failure — RS recovery, section 14); on success it reports `AssemblyFinished` and returns the file. Most honest receptions end right here, at the very first step.

8. Contested sectors: choice points

When a dispute exists, the assembler finds all **choice points**. For every list with ≥ 2 versions it looks at the "head": how many leading versions share the leader's counter. That number is the head size.

Version list	Head size	A choice point?
`[P×3, Q×1]`	1	no — the leader stands alone
`[P×3, Q×3]`	2	yes, iterate `P` and `Q`
`[P×2, Q×2, R×1]`	2	yes, iterate `P` and `Q`; `R` does not take part
`[P×1, Q×1, R×1]`	3	yes, iterate all three

Worth saying out loud: **only head versions take part in the search.** Minority versions (`R×1` in the third row) are not checked at all in the current assembly call. This is a deliberate trade-off: the search space stays small, and if the truth arrives again, its counter grows and it enters the head on its own.

9. How many combinations in total

The total number of combinations is the product of head sizes: `C = h₁ × h₂ × … × h_k`.

- one point with head 2 → `C = 2`;
- two points `3 × 2` → `C = 6`;
- three points `2 × 2 × 2` → `C = 8`;
- five points `10 × 10 × 10 × 10 × 10` → `C = 100,000` — right at the limit boundary (section 10).

If the product overflows a 64-bit integer it is clamped to `long.MaxValue`, and the progress message prints a "more than maximum" placeholder. Overflow never throws.

10. The fork: exhaustive search or rotation

Iterating all `C` combinations is the most reliable but also the most expensive route: every attempt means gluing a buffer and computing SHA-256. So before starting, the assembler does simple arithmetic:

1. **It tries candidate #1** (the zero combination) and **times** that attempt — `t₀`, down to `Stopwatch` ticks. If it succeeds, the matter is closed and no search is needed.
2. **It estimates the full pass:** `estimate = t₀ × C`.
3. **It decides by two conditions at once:**

Condition	Default threshold
not too many combinations	`C ≤ 100,000`
time estimate fits the budget	`t₀ × C ≤ 30 s`

Both hold → **exhaustive search** (odometer). Either fails → **heuristic rotation**.
A numeric example. The first attempt took 5 ms and there are 4,000 combinations → estimate `5 ms × 4000 = 20 s ≤ 30 s` → exhaustive. But if an attempt costs 10 ms with 6,000 combinations → estimate 60 s > 30 s → rotation: it is not obliged to cover everything, but it cheaply probes the "most plausible diagonals".

The time budget is **soft**: it is checked between attempts, never mid-attempt. Cancellation (the "Stop" button in the UI) is also checked between attempts.

11. Rotation: the "everyone advances one step" heuristic

Rotation first, because it is simpler. After every failed attempt each contested list advances by one element: **the first version of the head moves to the end of the head**. Minority versions below are untouched — so the descending-counter sort of the list is never broken.

The classic example: heads `A/B/C` (size 3) and `X/Y` (size 2). Start — candidate #1. Then each row is "shifted the lists → took the first elements → checked":

State	List 1	List 2	Selection
0 (start)	`A B C`	`X Y`	`A + X`
1	`B C A`	`Y X`	`B + Y`
2	`C A B`	`X Y`	`C + X`
3	`A B C`	`Y X`	`A + Y`
4	`B C A`	`X Y`	`B + X`
5	`C A B`	`Y X`	`C + Y`

All lists rotate in lockstep — hence "diagonals" are probed, not the full product. How many unique states are there? As many steps as it takes for **all** lists to return to their original order simultaneously — that is the least common multiple (LCM) of the head sizes: `LCM(3, 2) = 6`. In the example above rotation covered all 6 combinations — because 3 and 2 are coprime, the LCM equals the product.
Now heads `2 × 2 × 2`: the product is `C = 8`, but `LCM(2, 2, 2) = 2`. Rotation checks only `A+X+P` and `B+Y+Q`; the six "mixed" combinations (say, `A+Y+P`) are never seen. If the truth is among them — an honest refusal. That is the price of speed: a full pass would cost four times more.

The number of states is capped by the attempt limit (100,000 by default, counting the initial state). If the LCM exceeds it, rotation simply stops at the cap.

12. Exhaustive search: the odometer

The odometer works like a trip meter: every choice point gets its own "digit", the least significant one on the right. A step forward: the rightmost digit `+1`; when it reaches its head size, it resets to zero and `+1` carries into the neighbor to the left. When the carry walks off the leftmost digit, the combinations are exhausted.

Example for heads `3 × 2` (the senior digit is the three-element list, the junior — the two-element one). The zero combination has already been checked as "candidate #1"; the loop starts from the first:

Attempt	Odometer digits	Selection
0 (before the loop)	`(0, 0)`	`A + X`
1	`(0, 1)`	`A + Y`
2	`(1, 0)`	`B + X`
3	`(1, 1)`	`B + Y`
4	`(2, 0)`	`C + X`
5	`(2, 1)`	`C + Y`

Six attempts — **all** combinations, no gaps, no repeats. Between attempts, cancellation and the time budget are checked; when combinations or budget run out, `null` is returned.

Compare with section 11: on heads `3 × 2` both algorithms walk the same set of selections — the odometer row by row, rotation along diagonals.

13. Silent corruption and the volume subset search

Both strategies from sections 11–12 work when the damage is **visible**: a collision exists, so there is something to choose from. Now imagine the worst case: sector #7 arrived exactly once, its number is intact, its 72-bit sector hash matches — formally, an honest packet. But in reality the payload is corrupted: the medium "healed" a byte, or an attacker generated a packet with a correct hash right away. One version, no head, no choice point. Direct assembly and RS use volume #7 "as is", the final hash never matches, and there is nothing to iterate.

Stage 3 changes the question itself. Instead of "which versions to take" — "which volumes NOT to take". A suspicious volume is marked erased, as if it never arrived, and RS recovers its contents from the remaining volumes and ECC. Recovered, glued, checked against the final hash. That is the **volume subset search** — the last line of the search (we will call direct assembly and the version search stages 1–2).

Who gets excluded, and in what order, is decided by the planner:

1. **Base — all collision sectors at once.** If a sector has several versions and the previous stages could not sort them out — do not click through the versions; erase the whole sector and recover it from ECC. If even the base does not fit the ECC budget, the stage immediately admits defeat (an empty plan).
2. **Level 1 — one at a time.** Every present uncontested sector is declared erased in turn. The order is by suspicion: fewer confirmations — earlier; equal counts — in a random order (the randomness seed comes from H5, so the plan is identical for everyone and independent of the packet arrival order).
3. **Levels 2 and 3 — pairs and triples** from the 64 most suspicious sectors (the shortlist): what if two or three are corrupted at once.

A numeric example. `N = 64` data volumes, `M = 8` ECC volumes (the budget covers up to 8 losses). Only volume #7 is silently corrupted:

Step	Exclusion mask	What RS does	Outcome
stages 1–2	— (full map)	volume #7 counts as intact	hash mismatch
stage 3: base	∅ (no collisions)	—	straight to the levels
level 1, attempt 1	`{0}`	recovered #0 from the rest	hash mismatch
…	…	…	…
level 1	`{7}`	recovered #7 — the truth	**hash matches**

Every stage-3 attempt is a full RS recovery: first the expensive preparation (a Gaussian inversion, ~$E^2 \cdot N$ operations, where E is the number of erased data volumes), then the fast recovery. Hence the stage's two separate limiters: the **cap on E** (32 by default — above that a single attempt gets too expensive) and the pair "attempt count / time budget" (100,000 / 30 s). In the progress display the stage shows as "Volume subset search, attempt: k / estimate"; cancellation is checked between attempts.

Note: an ECC volume can be excluded too — suspicion does not depend on the role. True, every excluded redundancy volume reduces the recovery budget, and the mask feasibility limit accounts for that.

14. One attempt under the microscope

Both search kinds do the same thing at every step — they check the current selection. Inside an attempt there are two stages, strictly in order.

Stage 1 — direct assembly. Are all first `N` numbers of the selection filled (not `null`)? If even one data volume is missing, the attempt fails immediately. Otherwise: a buffer of `N × 64` bytes, payloads copied back to back, trimmed to `FileSize`, SHA-256 computed and compared to `H3` (details in section 15). Match — the file is done.

Stage 2 — RS recovery (only if an RS adapter was supplied and stage 1 failed):

1. Does the header declare `M > 0`? No (`M = 0`) — the attempt has definitively failed.
2. Build a presence map over all `T` numbers and count how many volumes (data + ECC) we have. There must be **at least** `N` — otherwise there is nothing to reconstruct from; failure.
3. Hand the received volumes and the map to the Reed–Solomon code; it returns `N` recovered chunks. If the decoder did not return `N` chunks, or any chunk is not 64 bytes — failure.
4. The chunks are glued into a buffer, trimmed, and verified against `H3` — exactly as in stage 1.

Why is direct assembly first? It is cheaper (no Reed–Solomon math) and in an honest, complete reception it almost always succeeds at once. RS is the fallback for genuinely lost volumes.

15. The final arbiter and memory hygiene: `TrimAndVerify`

The last bastion is a tiny but crucial function:

1. The `N × 64`-byte buffer is trimmed to `FileSize` (if the file size is not a multiple of 64, the last volume carries a "tail" of extra bytes — they are dropped).
2. SHA-256 of the result is computed and compared with `H3` from the header.
3. **Match** → the result goes up: the file is assembled.
4. **Mismatch** → both buffers (the full and the trimmed one, if they are different arrays) are **zeroed out** and `null` is returned. Why zero? So that "almost correct" data does not linger in memory and leak onward by mistake: a wrong candidate must not exist in any form.

16. Progress: what the user sees

Progress runs on a global 0…100 scale, throttled to whole percent steps; 100 is never shown until the file is truly assembled. Inside the search the counter advances by combinations, not bytes:

- search start: "Sector version search: ";
- exhaustive: "Exhaustive search: 4231 / 100000";

- rotation: "Version rotation: k / LCM";
- recovery: its own RS phase;
- finale: "Assembly finished" at 100 percent — only on success.

Cancellation (CancellationToken) is checked between attempts and throws outward — the UI correctly shows "cancelled by user" rather than "file did not assemble".

17. The slot's dashboard

Everything happening inside is observable from the outside — the UI uses these very metrics:

Metric	What it shows
`HeaderReceptionCount`	how many header copies were caught
`ReceivedSectorCount`	how many numbers are covered by at least one version
`ReceivedSectorCopyCount`	how many sector copies arrived in total (sum of all counters)
`CollisionSectorCount`	how many numbers have competing versions
`Coverage`	percentage of covered numbers out of `T`
`FormatValidityMap`	a string map: `▮` received, `▓` collision, `░` hole
`BuildCollisionMap`	a dictionary "number → how many versions"
`GetSectorVersions(n)`	a snapshot of one sector's versions in preference order

Example map for `T = 10`: the string `▮▮▓▮▮▮▮▮▓▓` reads as — 7 numbers covered (2 of them with collisions), 3 holes, coverage 70%.

18. Five study scenarios with numbers

18.1. Clean reception

A 100-byte file, no ECC: `N = ceil(100/64) = 2`, `M = 0`, `T = 2`. Both sectors received once. No choice points → one attempt: a 128-byte buffer → trim to 100 → SHA-256 matches `H3` → done. Progress: `AssemblyFinished` right away.

18.2. One contested sector

The same file, but sector #0 arrived in two variants: `A×3` and `B×3`. Head size 2 → one choice point, `C = 2`, the time estimate fits trivially → exhaustive search. Attempt 0 (candidate #1): `A` — hash mismatch. Attempt 1 (odometer `(0) → (1)`): `B` — match. The truth is found by the second attempt at the latest.

18.3. Two contested sectors: `3 × 2`

`C = 6`, `LCM(3,2) = 6`. If the time fits the budget, the odometer walks `(0,0) (0,1) (1,0) (1,1) (2,0) (2,1)`. If the time does not fit, rotation walks `A+X → B+Y → C+X → A+Y → B+X → C+Y`. The set of checked selections is **the same**; only the order differs (tables in sections 11 and 12). Both paths are complete.

18.4. A forgery won the counts

Sector #5: the truth `A` arrived 2 times, the forgery `B` — 5 times. The list `[B×5, A×2]` has a lone leader, no head → **no choice point**. Every selection takes `B`, the final hash never matches, assembly honestly refuses. The search is powerless: `A` is outside the search space (section 8). What does the system do? It keeps receiving: repeats will pull `A` up to a tie (`B×5, A×5`), and the next assembly call will see a choice point and start branching. Meanwhile a false file is never emitted — that is the whole point of the defense.

18.5. A silently corrupted volume

A file of 64 data volumes and 8 ECC volumes. Volume #7 is corrupted but passed the 72-bit sector check; one version, nothing to contest. Stages 1–2 take the volume "as is" — the hash mismatches. Stage 3 (section 13) excludes the present volumes one by one (level 1): on the mask `{7}` RS recovers the truth from the rest and the final hash matches. At worst a few dozen cheap attempts, each with E = 1.

19. Settings and limits

Every number named in this tutorial is a setting of the `SectorVersionSearchOptions` object:

Setting	Default	What it limits
`MaxExhaustiveCombinations`	100,000	the combination ceiling for exhaustive search
`MaxHeuristicAttempts`	100,000	the state ceiling for rotation (including the initial state)
`TimeBudget`	30 s	soft budget; checked between attempts

A separate group is the `SubsetSearch` property for stage 3 (section 13):

`SubsetSearch` setting	Default	What it limits
`MaxAttempts`	100,000	the stage's attempt ceiling
`TimeBudget`	30 s	the stage's soft budget
`MaxErasedDataVolumes`	32	erased data volumes per attempt (the cap on E)
`ShortlistSize`	64	the shortlist size for pairs and triples
`MaxExtraExclusionLevel`	3	the maximum size of extra exclusions

Rule of thumb: raise the limits — the chance of digging the truth out of deep combinations grows, but so does the worst-case time of a refusal. Lower them — the UI answers "does not assemble" faster, but

surrenders earlier more often. Counting overflows never throw: combinations are clamped to `long.MaxValue`, the LCM — to the attempt limit; the cap on E separately bounds the price of one stage-3 attempt.

20. What is guaranteed and what is not

Guaranteed (provable):

1. A file was emitted → its SHA-256 equals `H3` → it is the original, bit for bit (within the strength of SHA-256).
2. All data volumes received unambiguously → success on the very first attempt.
3. `C ≤ 100,000` and the time estimate fits → the correct combination within the heads will be found by exhaustive search (barring cancellation/budget — then an honest refusal).
4. Coprime head sizes → rotation covers all combinations, an equivalent of exhaustive search.
5. No false results ever: either a proven file, or `null`.

Not guaranteed (heuristics):

6. With common divisors among head sizes, rotation sees only the LCM diagonals.
7. Minority versions are not checked at all in the current call.
8. Fitting into 30 seconds is an estimate from the zero attempt's timing, not a promise.
9. A silently corrupted lone volume outside collisions will be recovered by stage-3 level 1 — provided the ECC budget and the limits hold.
10. Pairs and triples of silent corruptions are covered only within the shortlist and the exclusion level.

Numbers to memorize: sector hash — 72 bits, header hash — 192 bits, the arbiter — SHA-256; exhaustive search up to 100,000 combinations, rotation up to 100,000 states, subset search up to 100,000 masks with the cap on E = 32, budgets of 30 seconds each.

21. Self-check questions

1. Why is an ordinary sector repeat not a collision but a reinforcement of a version? (section 4)
2. From which header fields are `N`, `M`, and `T` derived? (sections 1–2)
3. Why can the assembly result never be "partially correct"? (section 3)
4. Why does a lower-count version not take part in the search, and how can it get in? (sections 8, 18.4)
5. By which rule is the choice between exhaustive search and rotation made? (section 10)
6. Why does rotation cover everything for heads `3 × 2`, but only a quarter for `2 × 2 × 2`? (section 11)
7. What happens to the buffers on a final hash mismatch, and why? (section 15)
8. Why is RS recovery performed after direct assembly and not before? (section 14)
9. What does the user see in the progress display during the search? (section 16)
10. Can a forgery become an assembled file? Why? (sections 3, 18.4)
11. Why does the volume subset search exclude whole volumes rather than versions, and who gets excluded first? (section 13)